

ACARA PRAKTIKUM 7

JARINGAN SYARAF TIRUAN 2

TUJUAN

- Menerapkan konsep JST dalam kode Python menggunakan TensorFlow dan Keras.

DASAR TEORI

1. Pengantar TensorFlow dan Keras

TensorFlow adalah *library open-source* yang dikembangkan oleh Google untuk melakukan komputasi numerik yang efisien, khususnya yang berkaitan dengan machine learning dan deep learning. TensorFlow menyediakan fungsionalitas untuk membangun, melatih, dan mengevaluasi model jaringan syaraf tiruan.

Keras adalah API tingkat tinggi yang berjalan di atas TensorFlow. Keras dirancang untuk memudahkan penggunaan TensorFlow dengan menyediakan abstraksi yang lebih mudah dimengerti. Dengan Keras, kita bisa membuat model *neural network* secara cepat dan sederhana.

Jalankan perintah berikut untuk menginstal TensorFlow dan Keras

```
pip install tensorflow
```

2. Layer

Layer adalah komponen utama dalam neural network, di mana transformasi data dilakukan. Layer umumnya dihubungkan secara berurutan, menghasilkan model neural network. Umumnya menggunakan *Dense layer*.

```
from tensorflow.keras.layers import Dense

# Membuat Dense Layer dengan 10 neuron dan fungsi aktivasi ReLU
dense_layer = Dense(
    units=10,
    activation='relu')
```

- 1) *units*: Jumlah neuron dalam layer. Setiap neuron akan menghasilkan output berdasarkan input yang diterimanya. Contoh: 1, 10, 50.
- 2) *activation*: Fungsi aktivasi yang digunakan untuk menambahkan non-linearitas pada output neuron. Contoh: *relu*, *softmax*, *sigmoid*, *tanh*, *none*

3. Pembuatan Model

Pembuatan model dilakukan dengan menyusun layer-layer. Salah satunya menggunakan Sequential, yaitu jenis model yang memungkinkan kita untuk menambahkan layer secara berurutan. Ini adalah cara yang paling sederhana untuk membangun model neural network.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Membuat model sequential dengan 2 dense layer
model = Sequential([
    Dense(10, activation='relu'),
    Dense(1, activation='softmax')
])

# atau bisa juga

model = Sequential()
model.add(Dense(10, activation='relu'))
model.add(Dense(1, activation='softmax'))
```

4. Compile

Fungsi compile digunakan untuk mengonfigurasi model sebelum dilatih. Ini mencakup pengaturan optimizer, fungsi loss, dan metrics yang akan dipantau.

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy'])
```

- 1) optimizer: Metode yang digunakan untuk memperbarui bobot model berdasarkan loss function. Contoh: adam, rmsprop, sgd.
- 2) loss: Fungsi yang digunakan untuk mengukur seberapa baik model dalam memprediksi output yang benar. Contoh: binary_crossentropy, categorical_crossentropy, sparse_categorical_crossentropy
- 3) metrics: Daftar metrik yang akan dipantau saat pelatihan dan evaluasi.

5. Pelatihan Model

Proses pelatihan melibatkan memberikan data input dan output ke model, sehingga model dapat mempelajari pola dan mengoptimalkan bobotnya. Model akan memperbarui bobotnya setiap epoch berdasarkan hasil prediksi, optimizer, dan fungsi loss.

```
# Melatih model dengan data latih
model.fit(
    X_train,
```

```
y_train,  
epochs=100,  
batch_size=32,  
validation_data=(X_val, y_val))
```

6. Evaluate

Fungsi evaluate digunakan untuk menilai kinerja model setelah dilatih. Ini memberikan informasi tentang nilai loss dan metrics yang ditentukan sebelumnya.

```
loss, accuracy = model.evaluate(X_val, y_val)
```

7. Predict

Fungsi predict digunakan untuk menghasilkan output dari model berdasarkan data input baru.

```
predictions = model.predict(X_new)
```

STUDI KASUS

Studi kasus ini berfokus pada klasifikasi spesies bunga Iris menggunakan jaringan syaraf tiruan (JST) dengan tujuan mengembangkan model yang dapat mengidentifikasi spesies bunga berdasarkan fitur morfologinya, seperti panjang dan lebar sepal serta panjang dan lebar petal. Dataset Iris terdiri dari 150 sampel dengan empat fitur dan satu label kelas yang mencakup tiga spesies: Iris Setosa, Iris Versicolor, dan Iris Virginica.

ALAT DAN BAHAN

1. Text editor/Google Colab/Kaggle Notebook/Jupyter Notebook

PERCOBAAN

Langkah-langkah:

1. Import *library* yang diperlukan.

```
import tensorflow as tf  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense, Input  
import pandas as pd  
import numpy as np  
from sklearn.preprocessing import LabelEncoder  
from sklearn.model_selection import train_test_split
```

2. Dataset Iris dimuat dari file CSV atau URL dan dipisahkan menjadi fitur (X) dan label (y). Fitur berupa empat kolom pertama, sedangkan label adalah kolom terakhir yang berisi jenis spesies bunga.

```
# Muat dataset iris dari file CSV/data
dataset =
pd.read_csv('http://archive.ics.uci.edu/ml/machine-learning-database
s/iris/iris.data', header=None, sep=',')

# Menyusun data X (fitur) dan y (label)
X = dataset.iloc[:, :-1].values # 4 kolom pertama sebagai fitur
y = dataset.iloc[:, -1].values # Kolom terakhir sebagai label
```

3. Konversi label yang berbentuk string menjadi bentuk numerik menggunakan LabelEncoder.

```
# Mengonversi label dari string menjadi numerik
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y) # Mengubah label jadi 0, 1, 2
```

4. Dataset dibagi menjadi data training dan testing. Data training akan digunakan untuk melatih model, sementara data testing dipakai untuk mengevaluasi performa model setelah pelatihan.

```
# Memisahkan dataset menjadi data latih dan data validasi dengan
rasio 80:20
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

5. Buat model neural network dengan 1 layer input dan 4 layer Dense. Layer input menerima fitur 4. Tiga layer awal menggunakan 1000, 500, dan 300 neuron dengan aktivasi ReLU, dan output layer menggunakan 3 neuron (karena ada 3 kelas) dengan fungsi aktivasi softmax.

```
model = Sequential([
    Input(shape=X_train.shape[1:]),
    Dense(1000, activation='relu'),
    Dense(500, activation='relu'),
    Dense(300, activation='relu'),
    Dense(3, activation='softmax')
])
```

Setelah membuat model, kita bisa menggunakan fungsi `summary()` untuk melihat `summary` dari arsitektur model yang telah kita buat.

```
model.summary()
```

6. Kompilasi model menggunakan optimizer Adam, loss categorical crossentropy (karena ini masalah klasifikasi multikelas), dan accuracy sebagai metrik.

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

7. Model dilatih menggunakan dataset training yang telah dibagi sebelumnya. Proses ini berlangsung selama beberapa epoch, di mana model secara otomatis belajar dari data untuk meminimalkan fungsi loss dan memperbaiki akurasi.

```
history = model.fit(  
    X_train, y_train,  
    epochs=50,  
    batch_size=32,  
    validation_data=(X_test, y_test)  
)
```

8. Setelah pelatihan selesai, evaluasi model pada data validasi untuk mendapatkan nilai loss dan accuracy.

```
loss, accuracy = model.evaluate(X_test, y_test)  
print(f"Loss: {loss}, Accuracy: {accuracy}")  
pd.DataFrame(history.history).plot(figsize=(10,6))
```

9. Kita dapat memvisualisasikan perubahan loss dan accuracy selama pelatihan untuk memantau kinerja model dari epoch ke epoch

```
pd.DataFrame(history.history).plot(figsize=(10,6))
```

10. Lakukan prediksi pada data baru (misalkan data validasi), dan tampilkan hasil prediksi beserta label aslinya.

```
predictions = model.predict(X_test)  
  
# Mengambil indeks dari nilai probabilitas tertinggi untuk setiap prediksi  
predicted_classes = predictions.argmax(axis=1)
```

```
print("Prediksi:", predicted_classes)
print("Label Asli:", y_test)
```

11. Kita dapat melihat distribusi prediksi yang benar dan salah di antara kelas-kelas yang ada. Confusion matrix menampilkan perbandingan antara prediksi (kolom) dan label asli (baris), dan ini membantu mengidentifikasi performa model pada setiap kelas.

```
# Buat confusion matrix
cm = confusion_matrix(y_test, predicted_classes)

# Visualisasikan confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=label_encoder.classes_, yticklabels=label_encoder.classes_)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```

12. Fungsi ini memungkinkan pengguna untuk memasukkan data baru, lalu memprediksi jenis spesies bunga berdasarkan data yang dimasukkan.

```
# Fungsi untuk memprediksi data input baru
def predict_new_data():
    sepal_length = float(input("Masukkan sepal length: "))
    sepal_width = float(input("Masukkan sepal width: "))
    petal_length = float(input("Masukkan petal length: "))
    petal_width = float(input("Masukkan petal width: "))

    # Membuat data array baru
    new_data = np.array([[sepal_length, sepal_width, petal_length,
                           petal_width]])

    # Melakukan prediksi
    prediction = model.predict(new_data)
    predicted_class = prediction.argmax(axis=1)

    # Mengonversi hasil prediksi numerik menjadi label asli
    predicted_label = label_encoder.inverse_transform(predicted_class)
    print(f"Prediksi kelas: {predicted_label[0]}")

predict_new_data()
```

TUGAS

Upload source code percobaan di atas ke GitHub dengan nama repositori sebagai berikut,

NIM-PraktikumKB-Pertemuan7

(Tugas diselesaikan di waktu praktikum sekarang juga)